

Squeezed Between Two Boundaries: Deterministic False Proofs and an Unseeing Metric in Metamorphic Testing for Object-Level Authorization

Qiulong Lin^{a,b}, Nanlei Chen^{a,b,*}

^a*Yango University, Fuzhou, 350015, China*

^b*Sogang University, Seoul, 04107, Republic of Korea*

Abstract

Metamorphic testing is the standard answer to the oracle problem in Web security testing: MST-wi defines 76 system-agnostic metamorphic relations (MRs) over a Web-specific domain language, and reports a specificity of 99.81%. We show that, for object-level authorization, these relations delegate adjudication to a *GUI-reachability model* that is a *crawl-time snapshot*, and that this delegation fails along a deterministic direction rather than as noise.

From a three-phase instantiation of the predicates as originally defined—crawl, out-of-band authorization change, execute—a 576-instance factorial study and a controlled study on Gitea 1.22.6, replicated on Gogs 0.14.3 and GitLab 17.11.7, we characterise the failure as three boundaries. (i) An access legitimately granted out of band after the crawl is reported as a *proven* violation—a *false proof*—because the predicate has only two representations and one is unavailable. (ii) Repairing this with a readable authorization oracle makes the MR step unnecessary in that oracle-available regime: replacing it with a single ordinary request changes nothing measurable, a descriptive, point-estimate comparison (not a formal non-inferiority test), and not domination. (iii) The reported specificity therefore rests on an undeclared premise—that the reachability model coincides with the true authorization state—and on an input-level denominator blind to scenario-level failure.

On Gitea the mismatch appears only under a crawl-time reading and never live, locating the mechanism in *time* rather than visibility; and the or-

*Corresponding author.

Email address: ubfbzu3@163.com (Nanlei Chen)

acle that would discharge the premise is readable from an owner’s vantage but never from a black-box subject’s own. We confirm the predicates’ semantics at source and execute the published relations—their source unmodified, with the build configuration and a replay harness added—reproducing the false proof on a real system. We contribute a characterisation, a measurement caveat, and one negative result.

Keywords: oracle problem, metamorphic testing, object-level authorization, BOLA/BFLA, specificity measurement, false proof

1. Introduction

Broken object-level authorization (BOLA), ranked first in the OWASP API Security Top 10 [1], is the failure to enforce, per object, that the authenticated subject is entitled to the object being touched. It is attractive to attackers precisely because the request is well formed and the endpoint is reachable: only the *entitlement* is wrong.

Detecting it is an *oracle* problem. A test that fires a request and reads the response cannot, by itself, tell a legitimate access from an illegitimate one, because a vulnerable system answers both with the same success status. The standard answer in the literature is *metamorphic testing*: instead of asserting an expected output, assert a relation that must hold across several executions. MST-wi [2] is the most developed instance of this idea for Web systems. It contributes a catalog of 76 system-agnostic metamorphic relations (MRs) written in a domain-specific language whose vocabulary contains 55 Web-specific predicates, and it reports that it “automatically detected 85% of their vulnerabilities” with a specificity of 99.81%.

This paper is about what happens to that claim when the object of the assertion is *object-level authorization*. Our starting observation is not a measurement but a reading of the relations themselves. To decide object-level authorization, an MR must know whether the acting subject is entitled to the object. MST-wi’s predicates do not express entitlement; they express what its own authors describe as reachability—what “the functionality exposed by the user interface” permits. The authors are explicit that this is a stand-in, and that the information it stands in for can be missing: to determine the test outcome, “we need a mapping between roles and resources, **which is not always available**” [2]. We take that sentence as our premise rather than our finding.

If the adjudication of an MR is delegated to a *reachability* model, then the model’s reading of the system—and in particular the moment at which it was read—becomes part of the specification. We therefore instantiate the predicates *as they are defined*, i.e. relative to a reachability model recorded at crawl time, and we drive the system through three phases: crawl, change authorization out of band, then execute the MR. The middle phase is the one the original design does not exercise, because in its evaluation the authorization state is stable across the crawl and the run.

Under that faithful instantiation we run a 576-instance factorial study over a constructed system, and a controlled study on Gitea 1.22.6. We do not propose a new mechanism; an earlier iteration of this work did, and our own ablation killed it (Section 4, Block 9). What survives is a characterisation of where the MR family stops being informative, stated as three boundaries: two that squeeze the technique from either side, and a third in the metric that would otherwise have reported it.

1. **The adjudication as designed produces deterministic false proofs.**

Taken as published—raw alarms, or alarms filtered by bookkeeping exceptions—it reports a legitimate access granted out of band after the crawl as an already-established violation: a *false proof* at rate 1.00 on the out-of-band scenario (8 of 8 alarms) and at rate 0.4000 on Gitea (10 of 25 authorized cells). The direction is deterministic rather than noisy, because the predicate has only two representations available and one of them cannot describe an out-of-band grant.

2. **When a sound, readable authorization oracle is available, the metamorphic step is not empirically necessary in our evaluated scenarios.** Repairing (i) by consulting a readable introspection surface works, but it is the *oracle* that does the work, not the relation: replacing the metamorphic step with a single ordinary request changes nothing measurable—false proofs 0 against 0, recall 0.50 against 0.49. We report *point-estimate non-inferiority*—a descriptive comparison, not a formal non-inferiority test (no margin was fixed in advance)—and not domination: the difference of 0.0139 is not distinguishable from zero (95% CI $[-0.0797, 0.1072]$). On the 3 records where the two methods disagree it is the MR-based method that errs, its reachability precondition blocking a detection the plain request makes.

3. **The reported specificity rests on an undeclared premise and on a low-resolution denominator.** The premise is that the reachability

model coincides with the true authorization state; the denominator is the count of generated *inputs*, which is structurally insensitive to a failure that is uniform across a whole scenario class.

The boundaries are not a claim that MST-wi is wrong. Two of them are anticipated in the original literature: the reachability stand-in is announced, and the predecessor design attributes its own residual false alarms to crawler coverage of “all URLs for all users” [3]. Our contribution is to locate the failure *mechanically*—it is a function of *when* the reachability model was read, not of *which* objects are visible—and to attach it to the reported metric.

Research questions. The characterisation is organised around three research questions, one per boundary. **RQ1** (temporal): does a crawl-time GUI-reachability proxy systematically misclassify authorisation changes that occur after the crawl? **RQ2** (oracle): when a sound, readable authorisation oracle is available, does metamorphic adjudication provide measurable benefit over a direct request that consumes the same observable? **RQ3** (measurement): can an input-level specificity metric reveal failures that are uniform at the scenario level?

Contributions.

1. A three-phase instantiation of MST-wi’s reachability predicates that makes the crawl-time reading explicit, together with two internal control methods that consume the same observables and therefore bound any claim about the MR step (Section 3).
2. A 576-instance factorial study (0 errors) whose design has 52 independent cells and 2 MRs, reported at that granularity (Section 4).
3. A controlled study on Gitea 1.22.6 in which the mismatch appears under a crawl-time reading (10/48) and never under a live reading (0/48), locating the mechanism in time rather than in visibility; and in which the oracle that would discharge the premise is readable from the owner/administrator vantage (56/56) but never from a black-box subject’s own vantage (0/40) (Section 5).
4. A replication of the same protocol on Gogs 0.14.3 and GitLab 17.11.7—one lineage-related, one independent—where the only readable oracles are readable but unsound, wrong on 17 (Gogs) and 18 (GitLab) of 48 cells, and the oracle-gated adjudicators convert those blind spots into 8 and 5 (v3) false proofs that the sound oracle of Gitea never produced (Section 5.8).

5. An execution of the library’s own authorization relations—their source unmodified, with the build configuration (`pom.xml`) and a replay harness added—against that system: the legitimate entitlement granted after the crawl is answered “cannot reach” 5 times and 2 relations raise an alarm on it, so the false proof is attributable to the published code rather than to our reading of it (Section 6.5).
6. One negative result: for this problem the MR step carries no work that a plain request does not carry, and we show it with a control that consumes the same observable quantity, rather than with a straw man (Section 4, Block 9).

Section 6 states the three boundaries and the condition under which the first of them may be lifted; Section 7 lists what our evidence cannot support.

2. Background: the predicates in question

2.1. *The oracle problem and metamorphic relations*

An oracle decides whether an execution has failed. For memory-safety or injection faults the oracle can often be a crash, a sanitizer, or a schema. For authorization there is no such intrinsic signal: the vulnerable and the correct execution are both successful HTTP responses. Metamorphic testing replaces the absent oracle with a relation over multiple executions: execute the same operation under a transformed configuration, and require that the two observed outcomes stay in a specified relation. The relation becomes the oracle.

The price is a new dependency. The relation is only as sound as the assumptions under which it is expected to hold, and those assumptions are, for authorization, exactly the facts that the absent oracle was supposed to supply.

2.2. *How MST-wi decides an authorization MR*

MST-wi [2] builds on the earlier metamorphic security testing design of Mai et al. [3]. It introduces a Security Metamorphic Relation Language (SMRL) with 55 Web-specific functions, and a catalog of 76 system-agnostic MRs. Its architecture has two stages: a crawler builds a model of the application as reached through the user interface and records, per user, which URLs and contents that user could reach; then a generator emits follow-up inputs

derived from a seed input and the MRs are checked against the recorded model.

The predicates that guard an authorization MR belong, in the artifact we inspected, to a small family. Two of them read the recorded reachability model directly—a “cannot reach through the GUI” predicate and a “user can retrieve content” predicate—and two read static properties of the acting subject, namely whether the subject is a supervisor of another or a site administrator. We say that such a predicate is *GUI-derived* when its value is a function of the crawl.

2.3. Two facts about the catalog, both counted mechanically

We parsed the authors’ public artifact rather than reasoning about the language in the abstract. Two counts matter for what follows.

The gate is GUI-derived, and the catalog is mostly not about authorization. Over the 76 MRs in the catalog, 38 (50.0%) gate on a GUI-derived predicate, and 40 gate on a GUI-derived or identity-property predicate. Only 10 MRs are named `OTG_AUTHZ` and only 9 call the credential-change action; their union is 14 (18.42%), of which 13 are gated on a GUI-derived or identity predicate. The honest reading of these numbers is a caveat, not a scandal: depending on the count, the authorization-relevant share of the catalog is between 10/76 and 14/76, and the union is an *upper bound* on authorization relevance, not a measurement. We report all counts together for exactly this reason, and we do not claim that authorization coverage is thin: the authors’ own accounting shows that they address 34 of 60 weaknesses under the *Authorize Actors* design principle, i.e. 57%, which ranks 3-rd of the principles they cover.

The vocabulary contains no authorization-artifact-state predicate. We searched the full text of the original paper for predicates that read the *state of an authorization artifact*—ownership, an access predicate, a permission record, a share. Words such as `isOwner`, `canAccess`, `ownerOf`, `hasPermission`, `ownership` and `shares_of` occur 0 times in total. This is not a discovery; it is the necessary consequence of the vocabulary being the one listed in the paper, and we treat it as a premise throughout. The predicates that *are* present occur with the counts 5/7/7/0 respectively, which is what makes the delegation to reachability unavoidable.

2.4. *Why this makes the reading moment part of the specification*

Suppose a subject’s entitlement to an object changes after the crawl: the owner grants it out of band, or the object is made public, or an administrative action grants access. Nothing in the two-stage architecture observes that change, because the model the MR consults was frozen at crawl time. The MR will then evaluate reachability against a world that no longer exists.

This is not a bug report about an implementation. It is a statement about what the relations *mean*: because they are defined over a recorded reachability model, their verdict is a statement about that model. Our experiments are constructed so that the two readings—the crawl-time one, and a live one—can be told apart, so that we can report which one is responsible for an observed disagreement.

3. Protocol: a faithful three-phase instantiation

3.1. *What “faithful” has to mean here*

This paper uses two instruments, and it matters which claim rests on which. The first is the three-phase instantiation described below; it carries the quantitative results of Sections 4 and 5. The second is a direct execution of the library’s own relations—their source unmodified, with the build configuration and a replay harness added—supplying the system configuration and crawl snapshot that the library does not ship; it carries one qualitative claim—that the published code, run, produces the false proof we predict—and is reported in Section 6.5.

Neither instrument is a benchmark. We report no sensitivity and no specificity attributable to MST-wi, and the run-level statements we do make are qualitative and confined to Section 6.5.

Faithfulness therefore has to be argued, and it has three parts. First, the predicates are evaluated against a reachability model *recorded before the MR executes*, because that is how the two-stage architecture defines them. Second, the adjudication follows the definition we give in Section 4.1: an MR fires, and the verdict may be an alarm, a pass, or an abstention. Third, and most importantly, the design admits a *control that consumes the same observable quantity as the MR*, so that the question “does the metamorphic step carry work?” can be answered without a straw man.

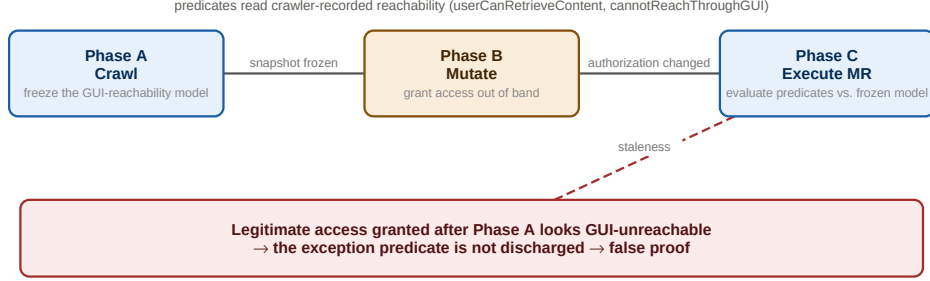


Figure 1: The three-phase instantiation. Phase 1 records the reachability model and freezes it; phase 2 changes an entitlement through a channel outside the crawled interface; phase 3 executes the MR, whose predicates consult the frozen model. The middle phase is the one the original design does not exercise, and it is where the reachability stand-in becomes wrong.

3.2. The three phases

Figure 1 shows the protocol.

1. **Crawl.** We record, for each subject and each object in the topology, whether the subject can reach the object through the interface, and freeze that record. This is the reachability model the predicates will consult.
2. **Change authorization out of band.** We alter the entitlement of a (subject, object) pair through a channel outside the crawled interface. The application’s own behavior is what changes; the frozen model is not updated, because the architecture has no mechanism to update it.
3. **Execute the MR.** We run the MR against the current system state while the predicates consult the frozen model.

Phase 2 is what the original evaluation does not exercise, and it is precisely where the reachability stand-in can be wrong. Figure 7 relates this to the boundaries of Section 6.

3.3. Adjudicator variants and internal controls

The MRs in question emit a raw alarm when the relation is violated. Turning a raw alarm into a verdict requires a policy for exceptions, and different policies are different methods. We therefore evaluate several:

- **v1 (raw).** Any raw alarm is reported as violation-proven.

- **v2 (exception-aware).** Known legitimate-sharing exceptions are filtered before reporting.
- **v3 (state-calibrated).** The adjudicator additionally consults an authorization introspection surface when one is readable, and abstains when it is not.
- **degrade_all.** The floor: abstain on every raw alarm. This method has 16 false proofs and 0.00 detection-keep rate, and no method is interesting unless it keeps some detection that **degrade_all** throws away.
- **direct_only.** Consume *only* an observable authorization surface, ignoring the access outcome. We include it to document a mistake: a control that ignores the access result is a straw man, and we show below that it is one.
- **direct_observe.** A plain request that succeeds, checked against the same readable authorization surface. This is the control that consumes the same observable quantity as the MR family, and it is the one that decides the negative result.

3.4. Scenario matrix and configuration axes

The scenarios are constructed to separate the reasons an MR can misfire:

- **S1, S2**—no shared object; control cases with no legitimate exception available.
- **S3**—a legitimate share created through the framework’s own sharing facility, so the share is visible in the framework’s log.
- **S4**—an unshared object; the plain violation case.
- **S5**—a read-only share exists, and the violation is a write: entitlement to read does not imply entitlement to write.
- **S6**—a legitimate share created *out of band*, so no framework log record exists.
- **S7**—an out-of-band share *plus* a genuine violation.

Each scenario is crossed with a configuration whose axes are the instantiation mode, whether the application enforces authorization correctly or violates it, the visibility of the sharing channel, and whether the authorization introspection surface is enabled. The full matrix is 576 records with 0 errors (576 usable), over 8 configuration labels.

3.5. Statistical treatment

We state the statistical basis here rather than in a footnote, because it constrains how every number in Sections 4 and 5 may be read.

The independent unit is the design cell, not the record. The 576 records are not 576 independent observations. They are the deterministic expansion of 52 design cells—pairs of (scenario, configuration)—crossed with 2 MRs, and, within a cell, with 3 exception-policy variants and 2 reachability readings, giving 6–12 records per cell. The two inner axes are switches we deliberately throw, not replications. Records inside a cell share the object graph, the setup and the ground truth.

Consequence for intervals. Every interval we report is a Wilson score interval (for a proportion) or a Newcombe hybrid-score interval (for a difference of proportions), computed by our own code in `compute_all.py`; we did not use `scipy`, which was unavailable in our environment. Because the design is a fixed enumeration rather than an i.i.d. sample, these intervals are *conventional* and systematically *narrow*. We therefore report, for every count, both the number of records and the number of design cells behind it, and we use the intervals to convey magnitude and direction only. We make no cross-method significance claim on the strength of a record-level interval.

Multiplicity. The comparisons in this paper are descriptive. We report no corrected family of tests, with one exception: a single prespecified contrast, the recall difference between `direct_observe` and `v3` (Section 4). A narrow interval makes a difference *easier* to declare significant, so the non-significance of that single contrast is robust to the optimism noted above.

Effective replication. The replication available to us is: one constructed system, 52 design cells, 2 MRs and one real system. The ceiling on any generalization to “MST-wi’s MRs” is therefore the number of MRs we instantiated, namely 2.

Small-denominator counts. Several of our headline counts have very small denominators—the anchored false-proof rate is $2/2$, whose interval $[[34.24\%, 100.0\%]]$ is wide by construction. Where a larger-denominator count of the same phenomenon exists we lead with that count instead, and we never present a small-denominator rate in the absence of its interval.

4. Synthetic study: where the adjudication goes

4.1. Definitions

The confusions in this paper are defined over the *verdict*, not over the MR’s trigger, because the trigger is evidence and the verdict is what a developer is told.

ground truth	verdict	counted as
violation	violation-proven	true positive
violation	not violation-proven (including abstention)	false negative
correct	violation-proven	false proof (FP)
correct	not violation-proven	true negative

A *false proof* is therefore the strict subset of false alarms that have been promoted to **violation-proven**: an error the tool asserts as established. We also report two rates that this table does not capture. The *detection-keep rate* is the share of true violations on which the MR fired and the method still alarmed; its denominator is restricted to true violations on which the MR fired, so that it measures whether a correction *discarded* a success rather than whether detection was possible. The *abstention count* counts only abstentions justified by an unreadable oracle, not cases where the MR simply did not fire.

4.2. The switch that cannot be thrown both ways

Table 1 crosses the visibility of the sharing channel against the correctness of the application’s authorization, on the representative slice.

Three of the four cells contain an error ($3/4$). One cell is clean—listed×correct, with no error at all—but it is clean for a trivial reason, and we say so: on that cell the MR never fires, so there is nothing for the adjudicator to get wrong, and there is no true violation to miss either. The same switch that makes it

Table 1: The 2×2 grid on the representative slice. FP = false proofs, FN = false negatives; each cell has 9 records over 52 design cells. Intervals are Wilson 95% and, per Section 3.5, are conventional rather than sampling-based.

channel	application	FP	FN	FP rate [95% CI]
unlisted	correct	4	0	44.44% [18.88, 73.33]
listed	correct	0	0	0.0% [0.0, 29.91]
unlisted	violating	4	0	44.44% [18.88, 73.33]
listed	violating	0	1	0.0% [0.0, 29.91]

clean makes the paired listed $\times$ violating cell err in the other direction, where the MR fails to fire on a genuine violation and produces 1 false negatives out of 4+1.

The substantive reading of Table 1 is therefore not “cells are error-free” but: *avoiding false proofs and keeping detection are controlled by one switch, and the switch cannot be thrown both ways at once*. On unlisted cells the false-proof rate is 44.44% with no false negatives; on listed cells the false-proof rate is 0.0% with false negatives appearing. Section 4.6 shows the same tension at the level of individual scenarios.

4.3. Attribution is clean, and the anchor is small

On the anchored configuration the false-proof rate is $2/2 = 100.0\%$, and this paper will never quote it without its interval, which is [34.24%, 100.0%]. The two controls that isolate attribution are clean: with the legitimate-sharing exception absent the rate is 0.00 (0/2), and with the channel listed the rate is 0.00 (0/2). The phenomenon is therefore attributable to the presence of a legitimate sharing exception that the MR cannot know about, not to the MR merely being executed. Recall on the unlisted violation scenarios S2, S4 and S5 is 1.00, 1.00 and 1.00 respectively.

Because 2 is two cases, we lead elsewhere with the larger counts: the per-cell 44.44% of Table 1 and, in Section 4.8, a 576-record panel.

4.4. Filtering exceptions does not help

One natural repair is to add exception predicates: recognise legitimate sharing and filter it. We evaluated the exception ladder—raw, then one exception level, then two—on the anchored false-proof rate. The rate is 1.00, 1.00 and 1.00: it does not move. This is a constructive result rather than a

Table 2: Observability of the sharing channel. Each scenario has 8 alarms over 52 design cells. A log record count of 0 means the framework has no evidence that the share exists.

scenario	framework log records	alarms	v2 false proofs	v
framework-created legitimate share (S3)	12	8	0	
out-of-band legitimate share (S6)	0	8	8	
out-of-band share with a real violation (S7)	0	0	0	

Errors are scenario-structured: bookkeeping-oracle FPs concentrate on legitimate out-of-band grants (S6)

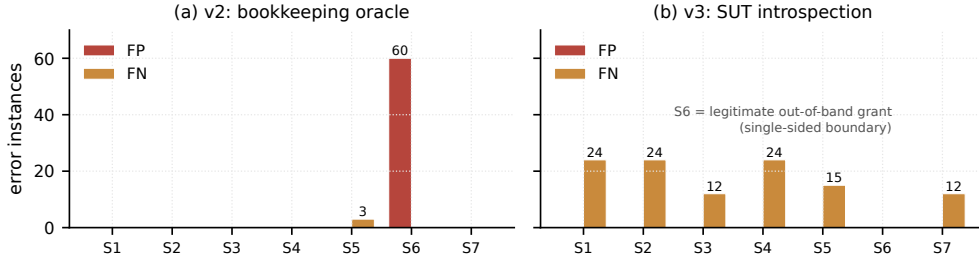


Figure 2: Errors are scenario-structured. The bookkeeping adjudicator’s false positives concentrate on the out-of-band legitimate-share scenario (S6), where the framework has no log record of the share, and the state-calibrated adjudicator’s false negatives concentrate where its reachability precondition blocks a detection. Every one of the 3 records on which `direct_observe` and `v3` disagree is of the second kind.

disappointing one: the exception levels we can express are themselves functions of the same reachability model, so they inherit its blind spot. Adding predicates from the same family cannot repair a failure caused by that family.

4.5. The framework cannot see the channel that matters

Table 2 separates the two channels. When the legitimate share was created through the framework’s own facility, the framework’s log holds 12 records on the scenario, and the exception-aware adjudicator `v2` produces 0 false proofs (rate 0.00). When the same legitimate share was created *out of band*, the framework’s log holds 0 records, and `v2`’s false-proof count rises to 8 of 8 alarms, i.e. a rate of 1.00, with every one of them promoted to violation-proven.

This is boundary 1 in its sharpest form.

The MR cannot distinguish “this share does not exist” from “this share exists but was not created through the interface I crawled”. Its only available representation of the first statement is the second, so an out-of-band grant is reported as a proven violation. Note also that on S7—where there *is* a genuine violation behind the out-of-band share—the same blindness would make the alarm right for the wrong reason: of the 6 true violations on that scenario, the exception-aware method reaches 6 and the state-calibrated method 4. Being right by accident is not detection, and we do not count it as such. Note also that on S7 the alarms and the true violations coincide numerically, which is exactly what makes the accident invisible to an input-level metric.

4.6. One switch, two scenarios

The grid’s tension can be exhibited inside a single MR, with no change other than the channel-visibility switch. Consider S3, where the correct behaviour is *not* to alarm (there is a legitimate share and no violation), and S5, where the correct behaviour *is* to alarm (a read-only share must not authorise a write). Under the unlisted configuration the MR is correct on S5 but wrong on S3; under the listed configuration it is correct on S3 but wrong on S5. Writing $c_x(S)$ for “configuration x is correct on scenario S ”: we obtain $c_{\text{unlisted}}(\text{S3}) = \text{False}$, $c_{\text{unlisted}}(\text{S5}) = \text{True}$, $c_{\text{listed}}(\text{S3}) = \text{True}$ and $c_{\text{listed}}(\text{S5}) = \text{False}$, so no configuration in this family is correct on both—the disjunction over configurations is False. For this MR the choice of switch is a choice of *which* error to make, not whether to make one.

4.7. The aggregation hides the regime change

Table 3 aggregates verdict quality over all configurations, using the recovery rate (share of raw alarms the method resolves correctly) and the detection-keep rate.

Read naively, v3 looks like the best method in the table: it recovers 1.00 of raw alarms while `degrade_all` keeps 0.00. That reading is wrong, and the way to see that it is wrong is to stratify. v3’s behaviour changes regime depending on whether an authorization introspection surface is readable, and the aggregate averages two different regimes:

stratum	records	false proofs	keep	abstentions
introspection readable	36	0	1.00	0
introspection absent	18	0	0.00	13

Table 3: Verdict quality over all configurations. “Recovery” is the share of raw alarms resolved correctly; “keep” is the share of fired true violations that still alarm. `degrade_all` is the floor that abstains on everything.

method	false alarms	recovery	keep
v1 (raw)	16	0.50	0.94
v2 (exception-aware)	16	0.50	1.00
v3 (state-calibrated)	16	1.00	0.49
<code>degrade_all</code>	16	1.00	0.00

Admissibility condition is measurable: false proofs vanish only while the introspection face is readable

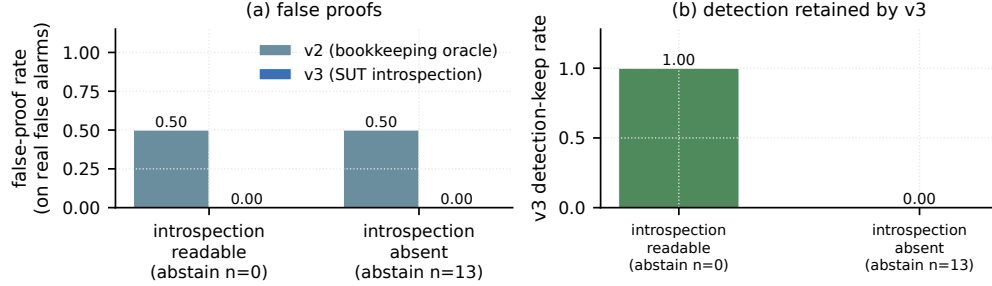


Figure 3: The same method, two regimes. Stratified by readability of the authorization introspection surface, v3’s false proofs vanish only on the stratum where it can still keep its detections; on the other stratum the zero false-proof count coincides with a keep rate of 0.00. Pooling the two strata (Table 3) converts abstention into apparent performance.

On the readable stratum v3 is genuinely good: it removes the false proofs the exception-aware method still produces (0 against v2’s 4) and still alarms on all 9 fired true violations. On the absent stratum its keep rate is 0.00: it alarms on none of the 5 true violations the exception-aware method still alarms on, and it abstains 13 times. The aggregate reports this as the best row in the table. It is not better; it is a different thing. A method that abstains on everything is not accurate, and any report that pools these two strata turns honest abstention into apparent performance. We therefore state, as a reporting rule, that v3’s numbers may appear only stratified.

Table 4: End-to-end confusion matrix over all 576 records (216 true violations, 360 correct cases). Intervals are Wilson 95% and are conventional (Section 3.5). `direct_only` is a deliberately unfair control and is shown to make that point.

method	TP	FP	FN	TN	precision	recall	F1
v1 (raw)	198	60	18	300	0.77	0.92	0.84
v2 (exception-aware)	213	60	3	300	0.78	0.99	0.87
v3 (state-calibrated)	105	0	111	360	1.00	0.49	0.65
<code>degrade_all</code>	0	0	216	360	—	0.00	—
<code>direct_only</code>	108	108	108	252	0.50	0.50	0.50
<code>direct_observe</code>	108	0	108	360	1.00	0.50	0.67

4.8. The decisive ablation: does the metamorphic step carry work?

Our earlier iteration proposed v3 as a mechanism, and the claim it rested on was that the metamorphic step was necessary to it. We tested that claim by removing the metamorphic step and replacing it with the most naive thing that consumes the same observable quantity.

The distinction matters. A first control, `direct_only`, consults the readable authorization surface but ignores whether the access succeeded; it is a straw man, and it shows: 108 false proofs against v3’s 0, precision 0.50. Rejecting `direct_only` proves nothing, because it discards half of the available evidence. The control that is actually fair is `direct_observe`: an ordinary request that succeeds, together with the same readable authorization surface. It consumes exactly what the MR-based adjudicator consumes.

Table 4 is the result that closes this line of work.

`direct_observe` has 0 false proofs against v3’s 0, with the same point-estimate precision (1.00), and its recall is 0.50 against v3’s 0.49. The recall difference of 0.0139 is not distinguishable from zero (95% CI $[-0.0797, 0.1072]$), so the correct description is a *descriptive, point-estimate* comparison, not domination; we do not claim a formal non-inferiority test (no margin was pre-specified), and the descriptive comparison is already enough to answer the question, because the two methods disagree on only 3 of the records. On every one of those, v3 does not fire the MR and `direct_observe` alarms on a genuine violation: the reachability precondition blocked a detection that a plain request finds.

We also report what a selective presentation would have hidden. v3’s precision of 1.00 is real, and it was bought with 111 false negatives; its F1

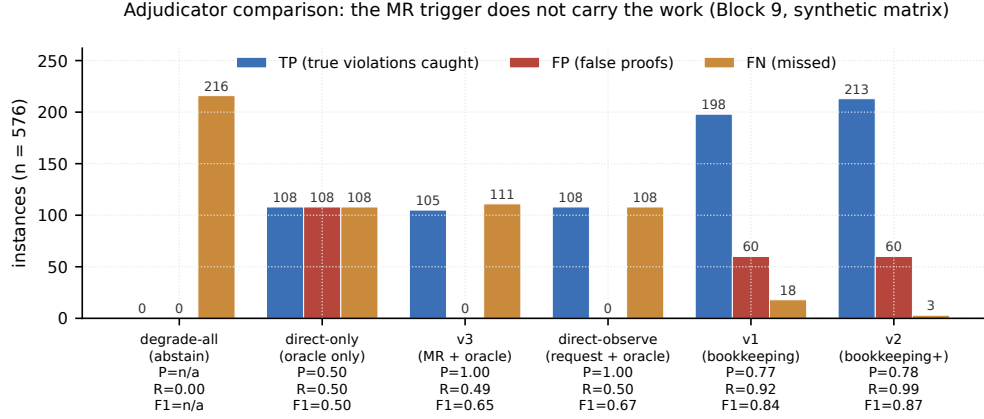


Figure 4: The MR trigger does not carry the work. Left: the deliberately unfair control `direct_only` compares badly, and rejecting it proves nothing. Right: the fair control `direct_observe`, which consumes the same observable quantity as the true label, attains the same point-estimate precision with 0 false proofs and a recall difference of 0.0139 (95% CI $[-0.0797, 0.1072]$, includes zero).

is 0.65. Quoting the precision alone, as our earlier iteration did, would have described a method that exchanges 60 false alarms for 111 missed violations as though it were an improvement. It is not, and it is not a mechanism that carries work: the metamorphic step adds nothing here that a single ordinary request does not already provide.

4.9. Reproducing the reported specificity, and why it cannot see this

The reported specificity has the form $1 - \text{false alarms} / \text{follow-up inputs}$. We cannot reproduce the engine’s exact input count, so we approximate the denominator as two follow-up inputs per case; this is enough to show the denominator’s resolution, which is the only point we wish to make. Table 5 gives the result.

The unlisted configuration—the one where a whole class of legitimate access is misreported—still yields a specificity of 0.846154 (84.62%). A denominator that counts generated inputs is divided by the number of requests the generator emitted, so a failure that is *uniform across a scenario class* is diluted by every other input in the run. This is boundary 3: the metric is not wrong, it is *low-resolution* with respect to the failure mode. It is also not our finding: the predecessor design attributes its own residual false alarms to crawler coverage of “all URLs for all users” [3], and *that* is the

Table 5: The reported specificity convention, approximated. The denominator is an input count, not a scenario count.

configuration	cases	false alarms	specificity
no legitimate-sharing exception	9	0	1.000000
exception, unlisted channel	13	4	0.846154
exception, listed channel	13	0	1.000000

same reachability-blindness we are describing, reported at a grain at which it cannot be seen.

A proposition on the metric’s resolution. The dilution above is not incidental; it is structural. Define the input-level specificity $S_{\text{input}} = 1 - FP_{\text{inputs}}/N_{\text{inputs}}$ and the scenario-level specificity $S_{\text{scenario}} = 1 - N_{\text{failed}}/N_{\text{scenarios}}$, where FP_{inputs} counts false alarms across all generated inputs and N_{failed} counts the scenarios in which the failure is uniform.

Proposition. Fix $f \geq 1$ failed scenarios, each producing a fixed number $a \geq 1$ of false-alarm inputs, and $s \geq 1$ successful scenarios each contributing m additional, alarm-free inputs. Then $S_{\text{input}} = 1 - (fa)/(fa + sm)$, which tends to 1 as $m \rightarrow \infty$, while $S_{\text{scenario}} = 1 - f/(f + s)$ is independent of m and strictly less than 1.

Proof. The total input count is $N_{\text{inputs}} = fa + sm$ and the false-alarm count is $FP_{\text{inputs}} = fa$; substitution gives $S_{\text{input}} = 1 - (fa)/(fa + sm)$. As $m \rightarrow \infty$ the fraction tends to 0, so $S_{\text{input}} \rightarrow 1$, whereas the scenario-level figure depends only on the fixed counts f and s . $\square$

The metric is therefore not merely imprecise: it is structurally blind to a failure mode that is uniform across a scenario class, and adding unaffected inputs cannot restore that resolution.

4.10. Second implementation

A second, independently written implementation of the same predicates agreed with the first on 54 of 54 records (1.00; Wilson 95% CI [0.934, 1.000]). This bounds implementation accident only. It does not bound a shared blind spot in the predicate semantics, because both implementations were written from the same reading of those semantics, and we say so rather than presenting the agreement as corroboration.

5. Real system: Gitea 1.22.6

5.1. Scope, and what this experiment can and cannot show

A synthetic system lets us construct the scenarios we want but cannot tell us whether the reachability reading we identified is an artefact of our construction. We therefore repeated the essential comparison on a real system.

We chose Gitea 1.22.6 because it has what the question requires: genuine object-level sharing with per-object visibility, a machine-readable sharing surface, and a documented administrative grant path. Two properties of the experiment bound its conclusions, and we state them before the numbers.

First, Gitea is *correct*: it does not contain the object-level authorization vulnerability we are studying. This experiment therefore measures only the false-proof side. It cannot measure detection, and we do not report a recall from it.

Second, the cell study below uses our instantiation of the predicates, not the engine. The engine’s own relations are executed separately, on the same system, in Section 6.5. We do not claim this experiment shows that MST-wi fails; we claim it locates where the adjudication departs from the system’s real authorization state—and Section 6.5 shows that the published code locates the same departure.

Third and most importantly, we report the reachability model under *both* readings—a crawl-time snapshot, which is what the original definitions presuppose, and a live reading—so that the reader can see which one is responsible. The experiment produces 96 cells, 48 under each reading.

5.2. Characterising the system

Table 6 characterises the grant surface. We counted an endpoint as a grant endpoint if, without modifying the system under test, a request to it can change a (subject, object) pair from not-entitled to entitled; the direction was determined by the HTTP methods actually available, so an endpoint offering only **DELETE** counts as a revocation, not a grant.

The counts above are raw grant-capable routes taken from Gitea’s API description (Swagger). The cross-vendor Table 10 reports a *normalised* M1 census in which semantically equivalent routes are merged and non-authorization grant surface (e.g. repository-transfer and star-toggle endpoints) is excluded; this is why its Gitea grant figure (13) differs from the raw route count (15)

Table 6: The grant surface of Gitea 1.22.6 (instance-independent).

quantity	count
paths in the system’s API description	253
grant endpoints	15
revocation endpoints	2
grant endpoints usable without site-administrator rights	8

reported here. The two tables answer different questions and are not in conflict.

The 8 endpoints that a non-administrator can drive are the ones that matter, because they are exactly the out-of-band channel of Section 4.5 in its real form: a subject who is not an administrator can create an entitlement that a crawl of that subject’s own interface does not have to reveal.

5.3. *Is the authorization oracle readable, and from where?*

Any claim that a readable oracle makes the metamorphic step unnecessary has to establish that the oracle is readable (and that its provenance is not the defect under test) for the party who would use it. We checked this deliberately from two vantages, because the answer differs.

From the owner/administrator vantage the sharing surface is readable on 56 of 56 attempts (56/56): complete. From a *black-box subject’s own* vantage—the vantage a black-box testing tool occupies—it is readable on 0 of 40 (0/40): never. The intermediate count of 16 of 56 readable attempts is not a counterexample, and we flag it explicitly because it invites a misreading: those successes come from subjects that happen to be the object’s owner, or a site administrator, and not from an arbitrary subject reading its own entitlements. The black-box count is the one that constrains the mechanism.

We verified that our model of the system agrees with the system’s own behavior: on 48 of 48 applicable cells the credential-substitution check agreed (8 cells were not applicable, being the case where the acting credential is the subject’s own).

5.4. *The mismatch is in time, not in visibility*

Table 7 is the load-bearing result of this section. Under the crawl-time reading, the adjudication departs from the real authorization state on 10 of 48 cells. Under the live reading it departs on 0 of 48. The same objects,

Table 7: Departure from the real authorization state, per reading of the reachability model, over the representative slice.

reading of the reachability model	mismatches	cells
crawl-time snapshot (as the definitions presuppose)	10	48
live at execution time	0	48

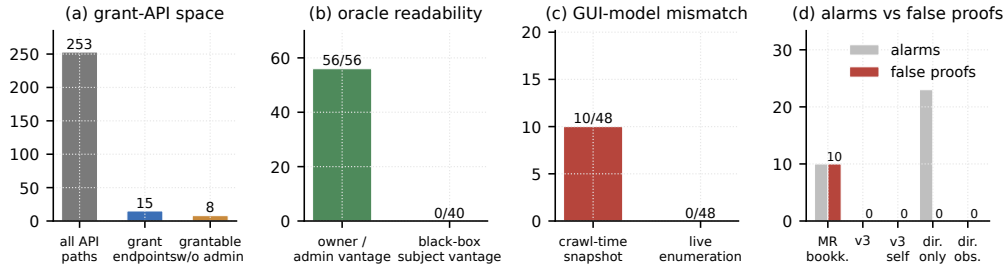


Figure 5: Gitea 1.22.6. (a) the grant-endpoint space; (b) readability of the authorization oracle by vantage—complete from the owner/administrator vantage (56/56), never from a black-box subject’s own vantage (0/40); (c) the mismatch is temporal: 10/48 under the crawl-time reading against 0/48 live; (d) alarms against false proofs per method.

the same entitlements and the same predicates produce a mismatch or not depending solely on when the reachability model was read.

This is what turns the synthetic result from a property of our construction into a property of the definition. On the synthetic system we could always be accused of having chosen objects that a crawler would miss; here, nothing about the objects changed between the two rows.

5.5. Consequences for the verdict

On the 25 cells that carry an authorized access, we evaluated the methods of Section 3 under the crawl-time reading. A bookkeeping implementation of the MR family produces 10 alarms, of which 10 are false proofs—a false-proof rate of 0.4000 on authorized cells. Under the live reading the same method produces 0 alarms. The state-calibrated adjudicator v3 produces 0 false proofs (rate 0.0000), consistent with its behaviour on the synthetic system when the oracle is readable.

The two controls behave as the synthetic study predicts, and one of them reproduces the straw-man failure on a real system: `direct_only` emits 23

Table 8: Provenance of authorized access, by observability channel.

channel	authorized cells
in-band share (visible in the framework log)	1
literal out-of-band grant	11
implicit: site-administrator or public-by-configuration	13
not observable by the framework log	24

alarms with 0 false proofs—its precision on this system is poor because it consults the sharing surface without checking whether the access actually succeeded—whereas `direct_observe` emits 0 alarms with 0 false proofs.

5.6. *How the authorization came to be, and why it is not a field rate*

Each authorized cell’s provenance was classified by channel (Table 8). Of 25 authorized accesses, 1 arrived through a share created in band and was therefore visible in the framework’s own records, while 24 were not observable by the framework’s log (0.9600); of those, 11 came through a literal out-of-band channel (0.4400).

We must be unambiguous here: these proportions are properties of the topology we constructed, not a field prevalence. Choosing a topology with many out-of-band grants is how we test the mechanism; it is not evidence about how often that happens in deployment. The instance-independent measurement in this section is Table 6; the channel breakdown is a design parameter. We label it as such wherever it is quoted.

5.7. *An endogenous control*

The most serious alternative explanation for the snapshot mismatch is that public objects are simply harder to enumerate, i.e. that the failure is about visibility rather than about time. We control for this with two objects that are public at the end of the experiment but public at different times (Table 9).

Both objects end in the same state: public. One was already public when the crawl ran, the other became public afterwards. The predicate reports that the subject cannot reach the object on 5 of 6 cells in the second case and 0 in the first; the MR fires on 5 cells in the second case and 0 in the first. Since the objects’ final visibility is identical, visibility cannot explain the difference. *Time* can, and does.

Table 9: Endogenous control: two objects that are both public at the end.

object	cells	<code>cannotReach</code> true	MR fires
public before the crawl	6	0	0
public only after the crawl	6	5	5

5.8. Cross-vendor replication: Gogs 0.14.3 and GitLab 17.11.7

Everything above was measured on one system. To test whether the measurements are properties of Gitea rather than of the design, we repeated the identical three-phase protocol (same 48-cell grid, same channel taxonomy, one shared pattern table normalising the M1 census) on two further systems (Table 10). **Gogs** 0.14.3 is the project Gitea forked from in 2016—related lineage, independently evolved code base; it publishes no swagger document, so its census denominator of 80 API paths comes from parsing the route table of the pinned version. **GitLab** 17.11.7 shares no lineage with either; its denominator of 1357 API paths comes from enumerating the routes of the pinned running instance (its own OpenAPI document is partial—core grant endpoints are absent from it—so it cannot serve as a denominator). The disclosure cuts both ways: Gogs is a replication across implementations, not across unrelated vendors; GitLab is a replication across vendors within one domain, the git forge.

The replication changes one thing and confirms two. The one thing it changes concerns the oracle. Gitea exposes a readable owner-vantage introspection surface (`GET .../collaborators/{u}/permission`) that is also *sound*: on all 48 cells its answer agrees with actual access. Gogs exposes only a collaborator-membership endpoint with 204/404 semantics. It is equally readable—48 of 48 cells at the owner vantage—but it is *unsound*: on 17 cells (35.4%) it reports no access for a subject who in fact has access. Its blind spot is exactly the set of authorizations that arise through any channel other than direct collaborator membership: site administration (8 cells), public visibility (5), and out-of-band grants (4). GitLab falls between the two, and the placement is mechanical rather than accidental. Its membership endpoint (`members/all/{u}`) resolves *inherited* membership by construction, so where Gogs’s oracle fails on team-derived access GitLab’s answers correctly (0 false negatives on that channel); but a membership oracle measures membership, and neither public visibility nor site administration confers membership. On those channels GitLab’s oracle is blind in its turn: 18 false negatives in all (8

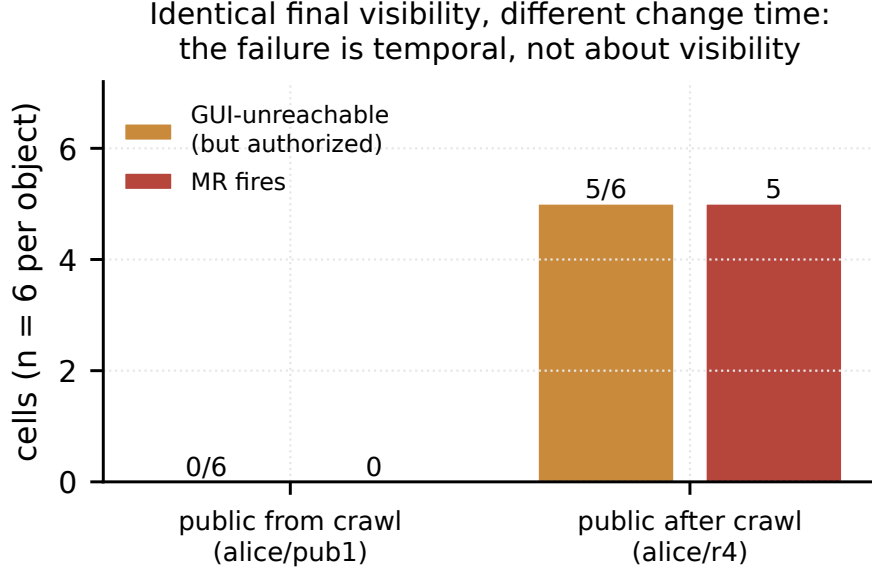


Figure 6: Endogenous control. Two objects are both public at the end of the experiment; one was public before the crawl, the other only after. Holding final visibility fixed, the predicate still reports unreachability on 5 of 6 cells only in the second case. The responsible variable is the time at which the reachability model was read.

administrative, 10 public), 37.5% of the grid. What the oracle cannot see is predictable from what the surface measures—which is exactly the judgement clause 2 of the admissibility condition asks for, and exactly why it cannot be automated from readability alone.

The consequence is measurable, and it is the strongest single datum in this section. On Gitea, the oracle-gated adjudicators produced no false proofs at all (0 for v3, 0 for direct-observe). On Gogs, the same adjudicators, fed the same design and the only oracle available, manufacture false proofs: 8 under the snapshot reading and 5 under the live reading for v3, and 17 for direct-observe. On GitLab they manufacture false proofs again—5 for v3, 18 for direct-observe—and the placement is surgical: every v3 false proof falls on a cell whose authorization arose through the visibility channel, the one channel the oracle cannot see. A readable oracle is not sufficient; a readable oracle with a blind spot does not merely fail to help—it converts its own blind spot into confirmed violations. On Gogs no admissible oracle exists;

Table 10: Cross-vendor replication under the identical three-phase protocol (crawl, out-of-band authorization change, adjudicate) on Gitea 1.22.6, Gogs 0.14.3, and GitLab 17.11.7. Counts are per reading of the same 48-cell grid; the live reading is given in parentheses where both exist. The snapshot reading is the operating point of the published relations. Gitea and Gogs share lineage (fork, 2016); GitLab shares none. The team-derived row is read out of the per-system oracle false negatives by channel (Section 5.8). The Gitea grant count here is the *normalised* M1 census (semantically equivalent routes merged, non-authorization grant surface excluded), and therefore differs from the raw route count in Table 6; the two tables use different normalisations and are not in conflict.

	Gitea 1.22.6	Gogs 0.14.3	GitLab 17.11.7
API entries in pinned version (M1 denominator)	253	80	1357
Grant endpoints (M1)	13	10	18
of which usable without site admin	10	4	17
Owner-vantage oracle readable	48	48	48
oracle agrees with actual access	48	31	30
oracle wrong (false negatives)	0	17	18
oracle wrong: team-derived access missed	0	4	0
Subject-vantage oracle readable	8	35	48
Reachability mismatch, snapshot (live)	10 (0)	19 (13)	10 (0)
False proofs: bookkeeping alarm, snapshot (live)	10 (0)	11 (5)	10 (0)
False proofs: v3 (oracle-gated), snapshot (live)	0 (0)	8 (5)	5 (0)
False proofs: direct-only (oracle only)	0 (0)	17 (17)	18 (18)
False proofs: direct-observe (oracle $\wedge$ request)	0 (0)	17 (17)	18 (18)

on GitLab none exists either; the honest adjudicator abstains on both.

Two results replicate across all three systems. First, the bookkeeping alarm false-proofs on legitimately granted access everywhere (10 cells on Gitea, 11 on Gogs, 10 on GitLab, snapshot reading), and the grant channels that cause it are the same kind: of the 56 recorded access grants on GitLab, 7 were out-of-band, 8 implicit through site administration, and 10 implicit through public visibility, against 1 in-band share—the same out-of-band-dominant profile as Gogs. Second, the reachability mismatch persists (19 of 48 cells on Gogs and 10 on GitLab under the snapshot reading, against 10 on Gitea). Two quantitative differences deserve note. On Gogs the mismatch only partially resolves under the live reading (13 of 48 remain), because objects granted through teams or public visibility never enter a subject’s personal crawl at all; on GitLab the membership-based listing surfaces group-derived access, so the live reading recovers everything the snapshot

Table 11: Positive control: detection of 3 deterministically seeded BOLA defects by each method. TP/FN/FP/TN are counted against the known seed, not against a model.

method	TP	FN	FP	TN
<code>mr_bookkeeping</code>	3	0	2	43
<code>v3</code> (owner vantage)	3	0	0	45
<code>v3</code> (subject vantage)	3	0	0	45
<code>direct_only</code>	3	0	30	15
<code>direct_observe</code>	3	0	0	45
<code>direct_observe</code> (subject)	3	0	0	45

reading missed (0 remain)—a second vendor-specific way for the snapshot premise to fail, this one in the crawl model rather than in the oracle. And GitLab’s mutation set includes a visibility flip that Gogs has no route for; its false-proof counts absorb that channel, which is why its direct-observe count (18) exceeds its v3 count while Gogs’s do not.

5.9. A positive control with a seeded vulnerability

Section 5 measures only the false-proof side, because the three real systems are correct and contain no object-level authorization vulnerability to detect. That leaves the question the reviewer is entitled to ask: *when a real object-level flaw is present, does the same detector find it?* We answer this with a positive control rather than by weakening any of the three systems in place.

We built a *controlled authorization target* that faithfully emulates the Gitea 1.22.6 object-level authorization API contract the engine actually consumes—the same endpoints and fields probed above—with a correct internal authorization model and 3 deterministically seeded BOLA defects. Each defect is a runtime bypass at `GET /api/v1/repos/{owner}/{repo}`: a subject who, by the model, has no entitlement (`none`) is nonetheless served the object. Crucially, the introspection surface (`GET /api/v1/repos/{owner}/{repo}/collaborators/{u}/perm`) reports the *correct* `none` permission, so the oracle stays sound and the defect is observable only through the discrepancy between what the oracle says and what the request returns. We drove the target with the *same* topology, mutation, crawl, introspection and four-method adjudication code as Section 5 (`run_real.py`), differing only in the system under test; the ground truth of which cells are vulnerable is supplied by the seed list, not inferred.

The observation-gated adjudicator behaves exactly as the negative result predicts (Table 11). `direct_observe` and `v3` both catch all 3 planted defects with zero false positives (TP = 3, FP = 0); they are not defeated by the presence of a real vulnerability. The straw man `direct_only` again overclaims, emitting 30 false positives on cells where the oracle says `none` but no access actually succeeds—the same failure it shows on the correct real systems. The paper’s central claim therefore now has both halves: when no vulnerability is present the metamorphic step adds nothing (§6), and when one is present the right adjudicator detects it without inventing others.

The control also delimits the detector. We seeded one BFLA defect—a non-privileged actor (bob) successfully performing a collaborator grant that only an owner or administrator should be able to (`PUT .../collaborators/{u}` returned HTTP 204, so the defect was present). The read-oriented introspection and observation logic does not cover this write-oriented action; that is a known boundary, consistent with the oracle-fidelity discussion in §6, and we report it as such rather than as a detection failure.

Honesty boundary, stated plainly: this is a controlled target, not production Gitea. It validates that the detector’s logic fires on a genuine broken object-level check and stays silent elsewhere; it does not claim to discover Gitea’s real CVEs. The system under test is a research artefact, and its source, seed list and run script are published with the rest of the reproducibility package.

6. Three boundaries

Figure 7 places the boundaries relative to the protocol. We state each one as a claim about the MR family, name the evidence that carries it, and say what would falsify it.

6.1. Boundary 1 (RQ1): the adjudication as designed produces deterministic false proofs

Taken as published—raw alarms, or alarms filtered by bookkeeping exceptions—the adjudication reports a legitimate access granted out of band after the crawl as an already-established violation, and it does so in one direction.

The evidence is twofold. On the constructed system, a legitimate share created out of band yields 8 false proofs out of 8 alarms, a rate of 1.00, against 0 when the same share was created through the framework’s facility. On Gitea, the bookkeeping adjudication produces 10 false proofs on 25

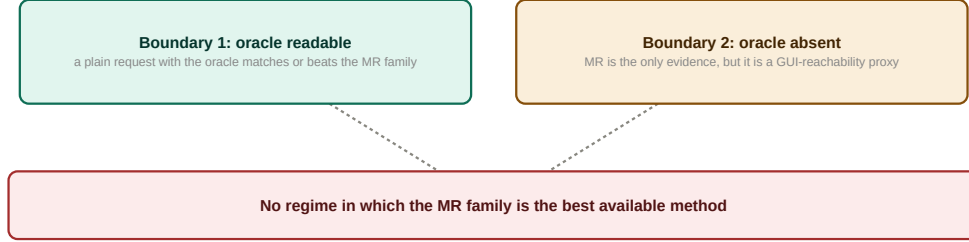


Figure 7: The three boundaries against the protocol of Figure 1. Boundary 1 is the deterministic false proof produced by the adjudication as designed; boundary 2 is the emptiness of the obvious repair—once a readable oracle exists, the metamorphic step is not empirically necessary; boundary 3 is the conditional nature of the specificity figure computed over the reachability model.

authorized cells (0.4000), and the same method under a live reading of the reachability model produces 0 alarms—so the error is not a property of the system, it is a property of the reading.

We stress the *deterministic direction*, because the original literature attributes its residual false alarms partly to asynchronous behaviour. Here the direction is forced. The predicate has exactly two representations available —“the subject cannot reach this object” and “the subject can reach this object”—and a grant created outside the crawled interface has no representation other than the first. The error is therefore systematic, which is why the false-proof count is not merely non-zero but equal to the alarm count on the out-of-band scenario. Section 7 records why our approximation of the exception semantics cannot turn this into a measurement of the original engine’s behaviour.

This boundary is not our discovery. The predecessor design attributes its own residual false alarms to the crawler failing to traverse “all URLs for all users” [3]. What we add is the identification of the failure as a function of *when* the model was read rather than *which* objects it contained, which is what makes the repair of Boundary 2 available— and, as we now show, empty.

6.2. *Boundary 2 (RQ2): once a sound, readable oracle exists, the metamorphic step is not empirically necessary*

If the false proofs of Boundary 1 are removed by consulting a readable authorization introspection surface, then it is the oracle that carries the work: replacing the entire metamorphic step with a single ordinary request changes nothing measurable.

The evidence is the ablation of Section 4.8. `direct_observe`—an ordinary request that succeeds, checked against the same readable surface—attains false proofs of 0 against v3’s 0, and recall 0.50 against v3’s 0.49. The Gitea experiment reproduces the ordering on a real system, with `direct_observe` emitting 0 alarms and 0 false proofs against v3’s 0.

Three qualifications are part of the claim, not disclaimers appended to it.

The first is that the false-proof comparison is a *tie at zero*, not a win. We state it as a tie because that is what it is: once either method can read the oracle, both stop producing false proofs, and the surviving difference is on recall alone. The pair of numbers in the previous paragraph must therefore be read as “no measurable change”, and a reader who takes 0 against 0 as an advantage has misread us.

The second is the honest arithmetic on that surviving difference. The recall difference is 0.0139 with 95% CI $[-0.0797, 0.1072]$, which includes zero, so `direct_observe` is *not* demonstrated to be better. It is demonstrated to be *not worse*, and that is already decisive, because for the metamorphic step to carry work it would have to be better at something the plain request cannot do. It is not: on the 3 records where the two methods differ, the MR-based method is the one that fails, its reachability precondition blocking a detection the plain request makes.

The third is that the control is *constructed*, not discovered. The two observables `direct_observe` consumes—the request succeeded, and the authorization surface reports no entitlement—are definitionally the same quantities that define the ground truth. The comparison is therefore a statement about the definitions, and it must be read as one: we do not claim a formal non-inferiority result, because no margin was fixed in advance; when the oracle is readable, the metamorphic relation has nothing left to compute. This is also why we do not present it as a competing method.

6.3. *Boundary 3 (RQ3): the reported specificity rests on an undeclared premise and a low-resolution denominator*

The reported specificity of 99.81% is a statement about a corpus of generated inputs under the assumption that the reachability model coincides with the true authorization state. It cannot detect a failure that is uniform across a scenario class, and that assumption is not stated.

There are two distinct issues and we keep them apart.

The premise. The adjudication of an authorization MR is a function of the reachability model (Section 2). A specificity figure computed over that adjudication is therefore conditional on the model being right. Where the model and the true authorization state diverge—exactly the out-of-band case of Boundary 1—the figure is not measuring the tool’s agreement with authorization, it is measuring its agreement with its own crawler. None of this is concealed by the authors: they say the role-to-resource mapping is “not always available” [2], and their predecessor names the crawler as the source of its false alarms [3]. Our point is that the number is reported without that condition attached.

The denominator. The published definition phrases specificity as the fraction of *generated inputs* that do not lead to a false positive—32 of 6401 in the predecessor’s own accounting, a figure whose own text attributes the residual cases to crawler coverage. A denominator in generated inputs is divided by every request the generator emits. A failure that removes an entire scenario class from correct handling contributes only as many false positives as that class emits inputs, so it is diluted by the rest of the run. In our approximation (Table 5) the configuration in which a whole class of legitimate access is misreported still reports a specificity of 0.846154. The metric is not computed incorrectly; it is *low-resolution with respect to this failure mode*, and that is a property of the choice of denominator, not of the arithmetic.

What we are not saying. We are not saying the reported figure is false, and we are not saying the reported false positives were misdiagnosed. The published attribution for those is asynchronous action, which is a different cause from the one we describe. The claim is narrower: a specificity in input-units, computed against a reachability model, cannot discharge the premise that model embeds, and cannot see a class-uniform failure.

6.4. Implementation-level confirmation

An earlier draft of this argument rested on the published *definitions* of the two predicates. Definitions can be read generously. We therefore obtained the implementation—the MST/SMRL library that realises MST-wi is public [2]—and checked the predicates at source. The library ships 23 concrete metamorphic relations, of which 11 are authorization relations; 8 of those gate on one of the two predicates we analyse (5 on `cannotReachThroughGUI`, 4 on `userCanRetrieveContent`), and 3 gate on neither.

Three facts emerged that the published text does not state.

The freeze is hard-coded, not incidental. `cannotReachThroughGUI` resolves to a set-membership test: for the current account it collects the URLs recorded in the *crawl* inputs and asks whether the URL under test is absent from that set. It issues no request. The per-account set is then cached in a field that is written exactly once and never invalidated anywhere in the library, so the snapshot is not an artefact of when the crawl happened to run; it is enforced by the implementation. `userCanRetrieveContent` is the same in kind: it compares against an output store read from a filesystem directory produced by the crawl, and that store too is loaded once and reused.

The clause that could have caught it is inert. The reachability test has a second clause: the URL must also be absent from the set recorded *across all users*, which is the clause that would prevent an alarm on an object many accounts can reach. That set is built by scanning every recorded sequence and stopping at the first login action; every record in our snapshot begins with a login action, so the set is empty and the clause never fires. This is not the same as the snapshot merely *having no second opinion*: the library *has* one and it cannot report, so the per-account record alone decides the verdict.

Missing data defaults to an alarm. The predicate returns `true`—“the subject cannot reach this object”—when the crawl input list is empty, and again when either argument is null. An empty crawl therefore *satisfies* the MR’s precondition instead of abstaining. This is the mechanism behind the deterministic direction we reported in Boundary 1: that failure is not noise, it is a default.

The role relation is configuration, not state. The guard on the second MR family is a supervisor relation read from the system’s static configuration file, and it reports `true` when the two accounts are the same. Attributes of

this kind are fixed at configuration time and cannot reflect an authorization granted afterwards.

What this removes is a specific objection—that we inferred the predicates’ semantics from prose. The semantics are in the code. It does not by itself establish that the code *behaves* as read, so we also executed it.

6.5. Executing the published relations

The library ships the relations but not the execution environment: no system configuration, no crawl snapshot, no subject system. We supplied those, and ran the relations themselves—source unmodified, with the build configuration (`pom.xml`) and an external harness/driver added—against Gitea 1.22.6.

What ran. 10 of the 11 authorization relations executed. The engine’s own input split partitions the input view into 160 parts, one per crawl record, and each part is evaluated against the 9 accounts it derives from those records’ login actions, so every relation is exercised over the full record–account product. 2 of the 10 relations returned at least one failure, 6 alarms in total. Those counts are the engine’s own `getFailures()` return value, not our adjudication of it. Table 12 lists them per relation.

What dominates the wall time. Almost none of the run’s 1483.7 seconds is analysis. Each of the 7520 leaf evaluations invokes the engine’s own browser-cleanup hook, which on Windows spawns two process-kill commands and waits on each whether or not a browser is present; relations that issue no request at all still consume minutes. This is a cost of the execution environment the artifact does not ship, and it bounds how much of the catalog a reproduction can exercise.

The load-bearing cell. Take the topology of Section 5: the subject bob holds a legitimate entitlement to `alice/s1`, granted out of band, after the crawl. On that cell the published predicate `cannotReachThroughGUI` answers “cannot reach”—which is the answer it is *defined* to give, because the subject’s crawl record contains no entry for the repository. Across the run the predicate was invoked 7321 times and returned that answer 1149 times (15.7%), of which 5 fall on this cell. 2 relations (`OTG_AUTHZ_002`, `OTG_AUTHZ_002c`) report a failure there. The alarm is therefore a false proof produced by the published relations, under the published predicate, on a real system: the subject’s request succeeds and returns the same bytes as the owner’s,

and the family’s guarded comparison—gate on the predicate, then compare the credentials-changed observation with the original (Section 6.4)—reads an unchanged observation as a violation. The access is authorized. Byte-equality is not inferred here from the lengths the run happened to record: the 3 granted cells were re-probed afterwards, and in each case the owner’s and the grantee’s response to the same URL hash identically while a third account is refused.

The alarms are not confined to that one cell, and they go nowhere else: the engine emitted 6 of them, on 3 cells, every one of them an object to which the subject holds a grant made after the crawl, and none on an object the subject is not entitled to.

What was and was not changed. We did not modify the library. `git status` on the upstream tree reports exactly 1 changed file, `pom.xml`, and 0 changed Java sources; the diff adds one `<source>` entry declaring the relation directory as a build source and removes nothing. Two classes were *added*: a provider and a driver, 612 lines together. The provider reproduces the session semantics the original replay relies on—the identity established by a login action carries to the actions that follow it, rather than being read off each action separately—because the credential-substitution helper rewrites only the credential-bearing action and leaves the rest carrying the original account. Getting this wrong does not merely add noise: it makes a credentials-changed input fetch under the original identity, so the observation is unchanged and the relation reports a violation on cells where the subject’s own objects are at stake. We record it because it is the kind of substitution defect that a false-proof rate cannot distinguish from the failure being studied. The substitution is otherwise confined to the observation channel, which the two-stage architecture makes separable: the relations read a crawl snapshot that the driver loads from disk, and `cannotReachThroughGUI` resolves to set-membership over those loaded records and issues no request of its own. Every record in that snapshot is re-read after the crawl as its own user—227 access points in all, each one resolving—which is necessary and not sufficient for a crawl record, but it rules out the failure mode that mattered here: a record no crawl could have produced. Our channel therefore cannot reach the predicate, which is why the cell above is attributable to the definition rather than to the harness.

Three limits belong to the claim. The content predicate `userCanRetrieveContent` was invoked 0 times, so this run does not exercise it; and because that pred-

Table 12: The published authorization relations (source unmodified; the build configuration and a replay harness were added) executed against Gitea 1.22.6. Alarms are the engine’s own return value. Requests counts observations actually issued, 270 in total: a relation evaluated on an output already observed earlier in the same run issues none, so requests are attributed to whichever relation observed them first.

MR	alarms	requests
OTG_AUTHZ_001	0	0
OTG_AUTHZ_001b	0	0
OTG_AUTHZ_001b2	0	0
OTG_AUTHZ_002	3	270
OTG_AUTHZ_002a	0	0
OTG_AUTHZ_002b	0	0
OTG_AUTHZ_002c	3	0
OTG_AUTHZ_002d	0	0
OTG_AUTHZ_002e	error	
OTG_AUTHZ_003	0	0
OTG_AUTHZ_004	0	0

icate is fed the observed output, our channel *would* propagate into it if it were—we report the count rather than assert a clean separation we did not test. The supervisor guard `isSupervisorOf` was invoked 8642 times (1321 true). Nor do all the relations gated on that predicate fire, and the reasons are inside their own guards rather than in the system: one compares the redirect URL a navigation produces, and our observations are direct responses with no navigation, so that attribute is empty and it passes; another requires a six-input composition our view does not furnish, so its guard is false and it passes. Nor does every relation that runs produce a verdict: 1 aborts with no request issued at all—the engine’s own reporting path dereferences an input slot its guard never bound—and that is the one missing from the executed count, shown as **error** in Table 12. We therefore read the alarms as originating in the gate, not as a count of how many published relations a full replay would flag. And Gitea is correct, so the run can only exhibit false proofs: it cannot measure detection.

6.6. The admissibility condition, and why it is the real ceiling

The results above suggest the only repair available inside this design. An adjudicator may promote a verdict to **violation-proven** on the strength of

a readable authorization introspection surface only when *both* of the following hold:

1. the application exposes a readable introspection surface, *and*
2. one can give a reason that this surface is not of the same kind as the defect under test—that is, that the surface is not itself a reachability or GUI-derived artefact whose blind spot is correlated with the blindness being tested for.

The second clause is the one that matters, and it is not automatable. It is a judgement about the provenance of a piece of evidence, and it must be made per system. The cross-vendor replication of Section 5.8 makes the clause concrete: on Gogs 0.14.3 the only readable surface fails it, on GitLab 17.11.7 the readable membership surface fails it on every channel that does not itself confer membership (it resolves inherited membership, and is blind to public visibility and site administration), and on both systems the oracle-gated adjudicators convert those blind spots into 8 (Gogs) and 5 (GitLab) false proofs for v3, where the sound oracle of Gitea produced none. This is why we describe the outcome as a boundary rather than as a mechanism: where the condition holds, the honest method is one that consumes the oracle directly and drops the relation (Boundary 2); where it does not, the honest method abstains, as v3 does on the unreadable stratum, where it keeps 0.00 of the detections the exception-aware method keeps and abstains 13 times. We note that in that stratum the aggregate table would have shown v3 as the best method in the study. It is not better; it has stopped answering.

6.7. *What would falsify each boundary*

- **Boundary 1.** A demonstration that the predicates are defined over live reachability rather than recorded reachability, which would remove the time-dependence entirely and make the out-of-band case unremarkable.
- **Boundary 2.** An authorization MR whose adjudication requires an observable that a succeeding request plus a readable oracle cannot produce—for instance, evidence about a subject’s entitlements that the subject itself cannot read but the test can. We do not have such an MR; we have two, of the same family (Section 7).
- **Boundary 3.** A specificity reported in scenario units, or with the reachability premise made explicit, that still fails to separate the configurations of Table 5.

7. Threats to validity

We inspected the implementation, and then executed it. Two limitations are easily conflated, so we separate them.

Source. The realisation of MST-wi is public [2]. We obtained it, read the predicates at source (Section 6.4), and built it under JDK 8 and Maven together with its 11 authorization relations, which load. Where we describe a predicate’s semantics we cite the code rather than the prose.

Execution. We then ran those relations—their source unmodified—against a real system; the only additions were the build configuration (`pom.xml`) and an external harness/driver that the upstream artifact does not ship (Section 6.5). This was not free: the library does not ship the per-system configuration and crawl data that its own demonstrations reference; nor does its build configuration compile the relations themselves, which live in a source directory the build file does not declare—a declaration we had to add, and Section 6.5 reports that one-line diff along with the rest of our footprint; executing a relation therefore required assembling a subject system, a system configuration and a crawl, which is what we did. The harness is ours, and its cost must be stated rather than assumed away: we execute without a browser, substituting the HTTP observation channel while leaving the relations, the predicates and the loaded crawl records untouched. That separation is exact for the reachability predicate, which is set-membership over loaded records and issues no request; it is *not* exact for the content predicate, which consumes the observed output, and that predicate happens not to be exercised by our data. We report its invocation count rather than claim a cleanliness we did not demonstrate.

What the run buys, and what it does not. It converts one class of claim: we can now say that a false proof is produced by the *published* relations under the *published* predicates on a real system, and not only by our reading of them. It does not convert the rest. The 576-record factorial and the cell study remain statements about our instantiation, because the harness is ours and because our exception vocabulary is drawn from the same predicate family: where we approximate an exception semantics we say so, and our result there is a statement about that family’s expressive limit, not a measurement of the engine’s false-positive rate. We still do not report a sensitivity or a specificity attributable to MST-wi, and we still do not assert that it fails.

Metric convention. The specificity of Section 4.8 uses two follow-up inputs per case as a stand-in for the engine’s input count. That approximation

is not the engine’s number, and we use it only to exhibit the denominator’s resolution. The conclusion we draw—that an input-unit denominator cannot see a scenario-uniform failure—does not depend on the exact count.

The independent unit is the design cell, not the record. As set out in Section 3.5, the 576 records expand 52 cells by 2 MRs and by two switches that we throw deliberately. Records inside a cell are not independent. All intervals in this paper are consequently conventional and optimistic, which makes our *negative* finding—non-significance of the recall difference in Boundary 2—conservative rather than fragile: a narrower interval only makes a difference easier to call significant, and we still cannot call ours.

Two MRs, and a catalog of one family. We instantiated 2 MRs, both from the same family—authorization relations whose gate is the absence of evidence. The native run of Section 6.5 executes 10 of the catalog’s 11 authorization relations, which lifts the count but not the family: 5 of them gate on the reachability predicate and 4 on the content predicate, and only the former produce alarms in our run, so the boundary we report remains a property of that gate. What the run broadens is the illustration of the boundary, not the evidence that the boundary is where we say. The ceiling on any generalization to “MST-wi’s MRs” therefore stands, and we do not generalize. This remains the single most important limitation of the paper, and it is the one we would address first with more work: raising it requires relations whose gate is *not* of this family, which is a different experiment from the one reported here.

Second implementation, shared blind spot. The second implementation agreed with the first on 54 of 54 records (1.00; Wilson 95% CI [0.934, 1.000]). Both were written from the same reading of the predicate semantics, so this bounds implementation accident only. It does not bound a shared misreading, and we do not present it as corroboration.

Real systems, all correct, none vulnerable. The real-system study spans three systems—Gitea 1.22.6, Gogs 0.14.3 (lineage-related: Gitea forked from Gogs in 2016), and GitLab 17.11.7 (no lineage)—chosen for accessibility rather than for representativeness, and none contains an object-level authorization vulnerability. The experiment therefore measures only the false-proof side and cannot speak to detection on a vulnerable real system. The replication is across git forges, one domain; it is not a sample of web applications at large.

Channel proportions are a design parameter. The breakdown of Table 8—including the 0.9600 share of authorized access that the framework’s log cannot see—reflects the topology we constructed to exercise the mechanism. It is not a field prevalence, and it must not be quoted as one. The instance-independent measurements in this paper are the grant-endpoint census (Table 6) and the predicate counts over the public artifact (Section 2).

Our own earlier claims are among the casualties. An earlier iteration of this work proposed the state-calibrated adjudicator v3 as a mechanism and reported its precision of 1.00. Two things were wrong with that. The precision was bought with 111 false negatives, so its F1 is 0.65 and quoting the precision alone was a selective presentation; and the control that we had used to argue for the mechanism’s necessity (`direct_only`, 0.50 precision) was a straw man because it discarded the access result. Replacing it with a fair control (`direct_observe`) is what produced the negative result of Section 4.8. We record this because it defines the condition under which our own claim would have been wrong, and a reader should demand that condition of any similar claim: a control that consumes the same observable quantity as the true label.

Constructed scenarios. The synthetic system is ours, its scenarios are designed to separate causes, and its ground truth is defined by construction. Section 5 exists precisely to test whether the result survives outside that construction, and it does, in the direction and with the endogenous control we report; but three systems, two of them related by lineage, tested once each, are still not a population.

8. Related work

Metamorphic security testing. The lineage this paper interrogates begins with metamorphic security testing for Web systems [3], which introduced system-agnostic relations comparing the behaviour of authorized and unauthorized users, and continues with the MST-wi design [2], which generalises it into 76 catalogued relations over a 55-predicate domain language. Our contribution is not a new relation but a characterisation of where this family’s adjudication stops discriminating, and of what the reported metric can and cannot see. We are explicitly building on the design’s own admission that the role-to-resource mapping is “not always available” [2].

Black-box and specification-driven API testing. RESTler derives producer–consumer dependencies from an API specification and prunes sequences with response feedback [4]. Search-based approaches extend white-box heuristics to REST APIs and to service interactions [5, 6]. These systems generate requests well but inherit the same oracle difficulty: a successful response cannot by itself distinguish an authorized from an unauthorized access. Our Boundary 1 describes what happens when the missing oracle is supplied by a reachability model rather than by the system.

Dedicated broken-access-control detection. A recent line targets access control directly, using black-box probing [7], access-policy-violation checks inside fuzzers [8], LLM-driven test generation for microservice authorization [9], automated detection of function- and object-level access-control flaws [10], and Java-specific access-control vulnerability detection [11, 12]. Database-level and policy-inference approaches recover the enforced access-control policy from the implementation, either to find missing owner checks [13, 14] or to extract the policy itself [15, 16, 17, 18]. Our question is orthogonal to all of these: we do not propose a better detector, we ask what an existing oracle-substituting technique actually decides when the entitlement information it needs is not among its observables.

Automated BOLA attack detection and the shape of the problem. Closely related work constructs the object-level attack and detects it [19], re-examines BOLA under a zero-trust framing [20], or evaluates broken access control empirically in deployed systems [21]. These establish that the problem matters and that it is empirically present; our contribution is a methodological measurement of one class of oracle substitution, not a new prevalence estimate.

LLM-based vulnerability detection and specification inference. Large language models are used broadly for vulnerability detection [22, 23, 24, 25, 26, 27, 28, 29, 30, 31], and there are efforts to extract privilege-related facts or program specifications with their help [32, 33], including work measuring the reasoning boundaries of LLMs on implicit specifications [34, 35]. Rows in this literature are also known to be of uneven quality [36]. We do not use an LLM in the detection loop: our negative result is an argument for keeping the *decision* in a deterministic component and being explicit about what that component can observe. An LLM-driven extension of this work would still

face Boundary 1, because the missing observable is missing regardless of who reasons about it.

The oracle problem as a first-class object. Finally, we note the methodological framing this paper belongs to. Studies that ask where a testing technique’s assumptions break—rather than proposing an improvement—are rarer than they should be, and in our own case the boundary was found by a control rather than by a hypothesis: the negative result of Section 4.8 was produced by replacing our own mechanism with the most naive consumer of the same observable. We regard the condition this exposes—that any claim of necessity must be tested against a fair substitute—as the transferable part of this work.

9. Conclusion

Metamorphic testing substitutes for a missing oracle by asserting a relation across executions. For object-level authorization the relation must know whether the acting subject is entitled to the object, and MST-wi’s predicates do not know that: they know what the interface, as crawled, permitted. We instantiated the predicates as defined—against a reachability model recorded at crawl time—and drove a constructed system through 576 records and a real system, Gitea 1.22.6, through 96 cells under two readings.

Three boundaries came out of it. Taken as designed—raw alarms, or alarms filtered by bookkeeping exceptions—the adjudication reports a legitimate access granted out of band after the crawl as a proven violation, at rate 1.00 on the constructed system and 0.4000 on Gitea; it does so deterministically, in one direction, because the predicate has two representations and an out-of-band grant fits only the wrong one. The obvious repair, consulting a readable authorization oracle, works—and, in that oracle-available regime, makes the metamorphic step unnecessary in our evaluated scenarios, because replacing it with a single ordinary request changes nothing measurable (false proofs 0 against 0, a tie at zero; recall difference 0.0139, 95% CI $[-0.0797, 0.1072]$, not distinguishable from zero), and on the 3 records where the two differ it is the MR-based method that fails. We report a descriptive, point-estimate comparison rather than superiority, and do not claim a formal non-inferiority test. On the real system the mismatch appears under the crawl-time reading (10/48) and never under a live reading (0/48), with an endogenous control that holds final visibility fixed; the responsible variable is

time, not visibility. And the reported specificity of 99.81% is conditional on a premise it does not state and is computed in units—generated inputs—that cannot resolve a scenario-uniform failure.

One methodological result deserves separating from the three boundaries. The arguments above could have been made—and in an earlier draft were made—from the published *definitions* of the two predicates, which is weak evidence, since a definition can be read generously. Reading the implementation instead (Section 6.4) confirms the semantics and adds two facts the published text does not state: the crawl-time model is frozen by a cache that is never invalidated, and a missing crawl makes the predicate report “unreachable”, so the MR’s precondition is satisfied by an absence of data rather than by evidence. That supplies a mechanism for the deterministic direction we report, and it is a mechanism that a definition cannot supply.

We did not set out to write this paper. Our first version proposed a mechanism, the state-calibrated adjudicator, and our own ablation removed it: once the metamorphic step is replaced by the most naive consumer of the same observable, the mechanism’s advantage disappears (0 false proofs against 0), and its precision had been bought with 111 false negatives. What is left is what we can defend: a characterisation, a measurement caveat, and one negative result.

What we do not claim. We do not claim that MST-wi is wrong, and we claim nothing about its accuracy as a detector. We read its engine (Section 6.4) and we ran it (Section 6.5); the run establishes a qualitative outcome, for one gate family, on one real system that is itself correct—not a rate. Its authors announce the reachability stand-in, and its predecessor attributes its own residual false alarms to crawler coverage. We do not claim a field prevalence for out-of-band grants; our channel proportions are a design parameter. We do not claim that our two MRs represent the catalog; they are two, of one family, and that is the ceiling on everything we say. And we do not claim to have found a better detector.

What we would do next, and what it would cost. We have taken the obvious first step: the library’s own relations now execute (Section 6.5), which lifts the false-proof side from our two relations to 10 of the catalog’s. That does not lift the ceiling that matters most. The run cannot measure detection, because the system is correct; and the relations it executes share one gate family, which is the thing we set out to characterise, so a broader catalog of

the same kind would broaden the illustration without testing the boundary. Two things would move this from a characterisation into something stronger, and both remain expensive. The first is a real system that *contains* an object-level authorization flaw, so that the detection side can be measured and not only the false-proof side. The second is relations whose gate is *not* the absence of evidence—checkable against a subject’s entitlements without pretending that entitlements are reachable facts—which is a construction problem, not a measurement problem.

The transferable part. The condition that killed our mechanism is the one we would put in front of any similar claim: a component’s necessity must be tested against a substitute that consumes the *same* observable quantity as the ground truth. A substitute that consumes less is a straw man, and a mechanism that survives only a straw man has not been shown to do anything.

Declarations

Data availability

The raw records of the three studies reported here—the factorial study, the real-system cell study, and the execution of the published relations—together with the scripts that compute every number in this paper from those records, were produced by the analysis pipeline described in Section 3. They are available from the corresponding author on request.

Competing interests

The author declares no competing interests, financial or otherwise.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Authors’ contributions

Both authors contributed to the work. Qiulong Lin and Nanlei Chen jointly developed the conceptualisation and methodology, designed and implemented the software and analysis pipeline, carried out the investigation and formal analysis, and wrote the manuscript.

Ethics

Not applicable. This study involved no human participants, no personal data, and no animals. All experiments were performed against software systems that we deployed and controlled locally.

References

- [1] OWASP Foundation, API1:2023 Broken Object Level Authorization, OWASP API Security Top 10, accessed 18 September 2026 (2023).
URL <https://api-security.owasp.org/editions/2023/en/Oxa1-broken-object-level-authorization/>
- [2] N. B. Chaleshtari, F. Pastore, A. Goknil, L. Briand, Metamorphic testing for web system security, *IEEE Transactions on Software Engineering* (2023) 3430–3471 doi:10.1109/tse.2023.3256322.
URL <https://doi.org/10.1109/TSE.2023.3256322>
- [3] P. X. Mai, F. Pastore, A. Goknil, L. Briand, Metamorphic security testing for web systems, in: 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), 2020, pp. 186–197. doi:10.1109/icst46399.2020.00028.
URL <https://doi.org/10.1109/icst46399.2020.00028>
- [4] V. Atlidakis, P. Godefroid, M. Polishchuk, Restler: Stateful rest api fuzzing, in: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), 2019, pp. 748–758. doi:10.1109/icse.2019.00083.
URL <https://doi.org/10.1109/ICSE.2019.00083>
- [5] A. Arcuri, M. Zhang, J. P. Galeotti, Advanced white-box heuristics for search-based fuzzing of rest apis, *ACM Transactions on Software Engineering and Methodology* (2024). doi:10.1145/3652157.
URL <https://doi.org/10.1145/3652157>
- [6] S. Seran, M. Zhang, O. Duman, A. Arcuri, Handling web service interactions in fuzzing with search-based mock-generation, *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3731558.
URL <https://doi.org/10.1145/3731558>

- [7] F. Liu, Y. Zhang, E. Li, W. Meng, Y.-Q. Shi, Q. Wang, M. Yang, Bacscan: Automatic black-box detection of broken-access-control vulnerabilities in web applications, in: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, 2025. doi:10.1145/3719027.3744825.
URL <https://doi.org/10.1145/3719027.3744825>
- [8] Ömür Şahin, M. Zhang, A. Arcuri, Enhancing rest api fuzzing with access policy violation checks and injection attacks, Journal of Systems and Software 243 (2027). doi:10.1016/j.jss.2026.113060.
URL <https://doi.org/10.1016/j.jss.2026.113060>
- [9] M. A. Uddin, S. Weerasinghe, C. Wojtak, T. Cerný, D. Silva-Junior, M. E. S. Ribeiro, A. S. Abdelfattah, Automated generation of microservice authorization tests using large language models, in: 2026 International Conference on Service-Oriented System Engineering (SOSE), 2026, pp. 41–50. doi:10.1109/sose71128.2026.00014.
URL <https://doi.org/10.1109/SOSE71128.2026.00014>
- [10] M. Schlaubitz, O. Veyisoglu, M. Rennhard, A2ct: Automated detection of function and object-level access control vulnerabilities in web applications, in: International Conference on Information Systems Security and Privacy, 2025, pp. 425–436. doi:10.5220/0013092700003899.
URL <https://doi.org/10.5220/0013092700003899>
- [11] Y.-Q. Shi, F. Liu, G. Yang, Y. Zhang, Y. Cao, E. Li, S. Chen, Facilitating access control vulnerability detection in modern java web applications with accurate permission check identification, IEEE Transactions on Information Forensics and Security (2025) 10683–10698doi:10.1109/tifs.2025.3614424.
URL <https://doi.org/10.1109/TIFS.2025.3614424>
- [12] R. Feng, M. Zhang, A. Xia, J. Cheng, Y. Li, Y. Zhang, Y. Liu, Jauthguard: Automatic detection for broken access control in java web apis, in: 2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2025, pp. 3102–3107. doi:10.1109/smc58881.2025.11343625.
URL <https://doi.org/10.1109/SMC58881.2025.11343625>

- [13] Y. Huang, C. Shi, J. Lu, H. Li, H. Meng, L. Li, Detecting broken object-level authorization vulnerabilities in database-backed applications, in: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, 2024. doi:10.1145/3658644.3690227. URL <https://doi.org/10.1145/3658644.3690227>
- [14] F. Liu, Y.-Q. Shi, Y. Zhang, G. Yang, E. Li, M. Yang, Mocguard: Automatically detecting missing-owner-check vulnerabilities in java web applications, in: 2025 IEEE Symposium on Security and Privacy (SP), 2025, pp. 903–919. doi:10.1109/sp61157.2025.00010. URL <https://doi.org/10.1109/SP61157.2025.00010>
- [15] H. Le, L. K. Shar, D. Bianculli, L. Briand, D. C. Nguyen, Automated reverse engineering of role-based access control policies of web applications, J. Syst. Softw., pp. 111109 (2021). doi:10.1016/j.jss.2021.111109. URL <https://doi.org/10.1016/j.jss.2021.111109>
- [16] B. Berger, R. W. Nguemphang, K. Sohr, R. Koschke, Static extraction of enforced authorization policies seeauthz, in: 2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation (SCAM), 2020, pp. 187–197. doi:10.1109/scam51674.2020.00026. URL <https://doi.org/10.1109/SCAM51674.2020.00026>
- [17] W. Zhang, D. Bali, J. Kerney, A. Panda, S. Shenker, Extracting database access-control policies from web applications, ArXiv (2024). doi:10.48550/arxiv.2411.11380. URL <https://doi.org/10.48550/arXiv.2411.11380>
- [18] C. Zuo, Q. Zhao, Z. Lin, Authscope: Towards automatic discovery of vulnerable authorizations in online services, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017. doi:10.1145/3133956.3134089. URL <https://doi.org/10.1145/3133956.3134089>
- [19] A. S. Filho, R. J. Rodríguez, E. Feitosa, Automated broken object-level authorization attack detection in rest apis through openapi to colored petri nets transformation, International Journal of Information Security (2025). doi:10.1007/s10207-024-00970-5. URL <https://doi.org/10.1007/s10207-024-00970-5>

- [20] A. Wu, Z. Feng, R. Feng, Z. Xing, Y. Liu, Rethinking broken object level authorization attacks under zero trust principle, *ACM Transactions on Software Engineering and Methodology* (2026). doi:10.1145/3833417. URL <https://doi.org/10.1145/3833417>
- [21] S. E. H. Chehade, F. Hantke, B. Stock, 403 forbidden? ethically evaluating broken access control in the wild, in: *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 3218–3235. doi:10.1109/sp61157.2025.00252. URL <https://doi.org/10.1109/SP61157.2025.00252>
- [22] W. Guo, Y. Fang, C. Huang, H. Ou, C. Lin, Y. Guo, Hyvuldect: A hybrid semantic vulnerability mining system based on graph neural network, *Computers & Security* (2022). doi:10.1016/j.cose.2022.102823. URL <https://doi.org/10.1016/j.cose.2022.102823>
- [23] G. Lu, X. Ju, X. Chen, W. Pei, Z. Cai, Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning, *Journal of Systems and Software* (2024). doi:10.1016/j.jss.2024.112031. URL <https://doi.org/10.1016/j.jss.2024.112031>
- [24] Y. Yang, X. Zhou, R. Mao, J. Xu, L. Yang, Y. Zhang, H. Shen, H. Zhang, Dlap: A deep learning augmented large language model prompting framework for software vulnerability detection, *Journal of Systems and Software* (2024). doi:10.1016/j.jss.2024.112234. URL <https://doi.org/10.1016/j.jss.2024.112234>
- [25] Q. Mao, Z. Li, X. Hu, K. Liu, X. Xia, J. Sun, Towards explainable vulnerability detection with large language models, *IEEE Transactions on Software Engineering* (2025). doi:10.1109/tse.2025.3605442. URL <https://doi.org/10.1109/tse.2025.3605442>
- [26] X. Yin, C. Ni, S. Wang, Multitask-based evaluation of open-source llm on software vulnerability, *IEEE Transactions on Software Engineering* (2024). doi:10.1109/tse.2024.3470333. URL <https://doi.org/10.1109/tse.2024.3470333>

- [27] M. Ehrenberg, S. Sarkani, T. A. Mazzuchi, Python source code vulnerability detection with named entity recognition, *Computers & Security* (2024). doi:10.1016/j.cose.2024.103802.
URL <https://doi.org/10.1016/j.cose.2024.103802>
- [28] M. C. Fu, C. Tantithamthavorn, T. Le, Y. Kume, V. Nguyen, D. Phung, J. Grundy, Aibug Hunter: A practical tool for predicting, classifying and repairing software vulnerabilities, *Empirical Software Engineering* (2023). doi:10.1007/s10664-023-10346-3.
URL <https://doi.org/10.1007/s10664-023-10346-3>
- [29] J. Liu, G. Lin, H. Mei, F. Yang, Y. Tai, Enhancing vulnerability detection efficiency: An exploration of light-weight llms with hybrid code features, *Journal of Information Security and Applications* (2024). doi:10.1016/j.jisa.2024.103925.
URL <https://doi.org/10.1016/j.jisa.2024.103925>
- [30] J. Senanayake, H. Kalutarage, A. Petrovski, L. Piras, M. O. Al-Kadri, Defendroid: Real-time android code vulnerability detection via blockchain federated neural network with xai, *Journal of Information Security and Applications* (2024). doi:10.1016/j.jisa.2024.103741.
URL <https://doi.org/10.1016/j.jisa.2024.103741>
- [31] A. Mechri, M. A. Ferrag, M. Debbah, Secureqwen: Leveraging llms for vulnerability detection in python codebases, *Computers & Security* (2024). doi:10.1016/j.cose.2024.104151.
URL <https://doi.org/10.1016/j.cose.2024.104151>
- [32] H. Wang, Z. Wang, P. Liu, A hybrid llm workflow can help identify user privilege related variables in programs of any size, *ArXiv* (2024). doi:10.48550/arxiv.2403.15723.
URL <https://doi.org/10.48550/arXiv.2403.15723>
- [33] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, C. Tian, Enchanting program specification synthesis by large language models using static analysis and program verification, in: *International Conference on Computer Aided Verification*, 2024, pp. 302–328. doi:10.1007/978-3-031-65630-9_16.
URL https://doi.org/10.1007/978-3-031-65630-9_16

- [34] R. Wang, H. Zhuang, H. Cao, S. Chen, Measuring the reasoning boundaries of large language models for implicit security invariants in code: A controlled empirical study, in: 2026 IEEE 50th Annual Computers, Software, and Applications Conference (COMPSAC), 2026, pp. 3146–3157. doi:10.1109/compsac69091.2026.00467.
URL <https://doi.org/10.1109/COMPSAC69091.2026.00467>
- [35] T. Le-Cong, B. Le, T. Murray, Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference, in: Annual Meeting of the Association for Computational Linguistics, 2025, pp. 21991–22014. doi:10.18653/v1/2025.acl-long.1068.
URL <https://aclanthology.org/2025.acl-long.1068/>
- [36] R. Croft, M. A. Babar, M. M. Kholoosi, Data quality for software vulnerability datasets, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), 2023, pp. 121–133. doi:10.1109/icse48619.2023.00022.
URL <https://doi.org/10.1109/ICSE48619.2023.00022>